Single-Program LangGraph Agentic AI Lab

Cohere + RAG + Web Search + Calculator + Reviewer + Human Approval

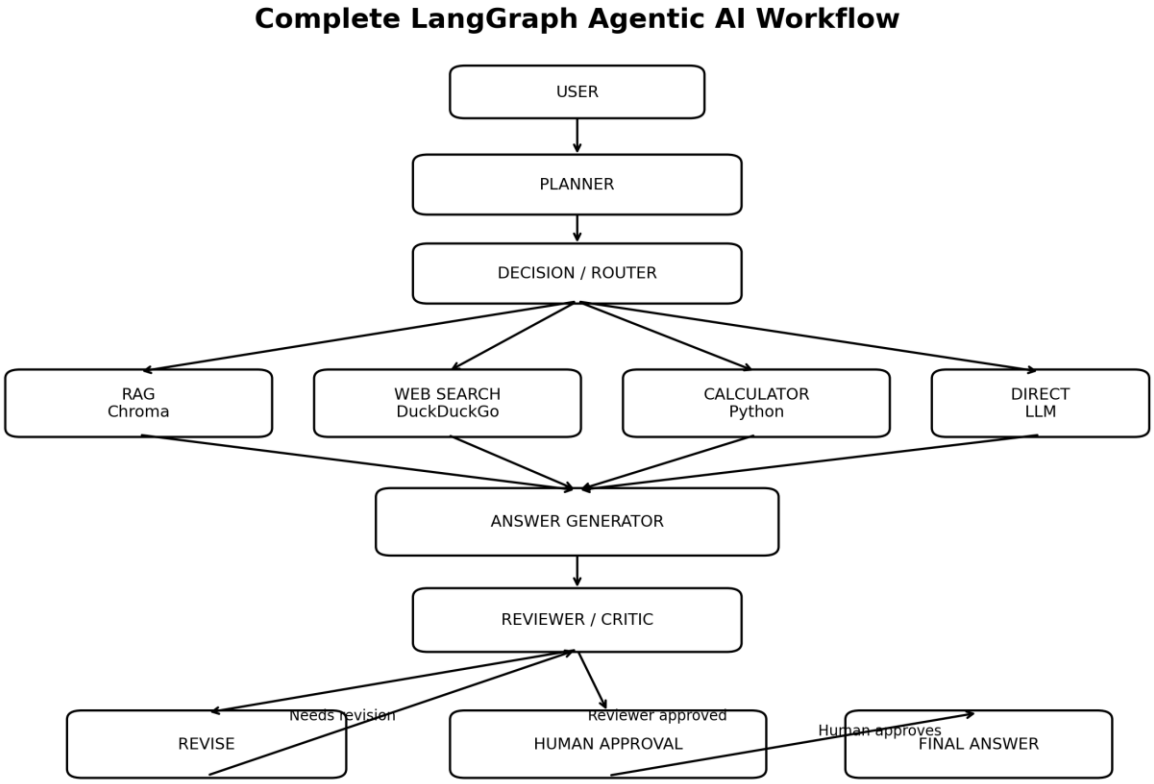
Hands-on Teaching Guide

1. Lab Objective

This lab combines the complete LangGraph Agentic AI workflow into a single Python program. Students can observe planning, routing, RAG, web search, calculation, answer generation, review, revision, human approval, and final answer generation in one executable file.

- Use Cohere as the reasoning LLM.
- Use Chroma and Cohere embeddings for RAG over a PDF.
- Route dynamically among RAG, web search, calculator, or direct reasoning.
- Generate a draft answer and review it with a Critic/Reviewer node.
- Loop through revision when necessary.
- Pause for human approval before producing the final answer.
- Maintain graph state using a LangGraph checkpointer.

2. Complete Workflow Diagram



The graph coordinates specialized stages and can branch, loop, pause, and resume. This is the key conceptual jump from a simple LLM response pipeline to an agentic workflow.

3. Project Structure

```
langgraph-agentic-ai/
│
```

```
├── documents/
│      └── company_policy.pdf
│
├── chroma_db/
│
└── langgraph_complete_agent.py
```

4. Install the Required Packages

```
pip install -U langgraph langchain langchain-community \
langchain-cohere langchain-chroma langchain-text-splitters \
pypdf duckduckgo-search
```

Set the Cohere API key in Windows Command Prompt:

```
set COHERE_API_KEY=your_cohere_api_key
```

PowerShell:

```
$env:COHERE_API_KEY="your_cohere_api_key"
```

5. Complete Single Python Program

```
# ============================================================
# COMPLETE LANGGRAPH AGENTIC AI PROGRAM
# ============================================================

import os
import re
from typing import TypedDict

from langchain_cohere import ChatCohere, CohereEmbeddings
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_text_splitters import RecursiveCharacterTextSplitter

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command


PDF_FILE = "documents/company_policy.pdf"
CHROMA_DIRECTORY = "./chroma_db"
COLLECTION_NAME = "company_documents"
THREAD_ID = "classroom-demo-001"


if not os.getenv("COHERE_API_KEY"):
    print("\nERROR: COHERE_API_KEY is not configured.")
    print("Command Prompt: set COHERE_API_KEY=your_cohere_api_key")
    print('PowerShell: $env:COHERE_API_KEY="your_cohere_api_key"')
    exit()


llm = ChatCohere(
    model="command-a-03-2025",
    temperature=0
)

embeddings = CohereEmbeddings(
    model="embed-v4.0"
)


def initialize_vector_database():

    print("\nInitializing knowledge base...")

    if os.path.exists(PDF_FILE):

        loader = PyPDFLoader(PDF_FILE)
        documents = loader.load()

        print(f"Pages loaded: {len(documents)}")

        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=800,
            chunk_overlap=150
        )
```

```
        chunks = text_splitter.split_documents(documents)

        print(f"Chunks created: {len(chunks)}")

        vector_store = Chroma.from_documents(
            documents=chunks,
            embedding=embeddings,
            collection_name=COLLECTION_NAME,
            persist_directory=CHROMA_DIRECTORY
        )

        print("Vector database ready.")

    else:

        print("PDF not found. Connecting to existing Chroma database...")

        vector_store = Chroma(
            collection_name=COLLECTION_NAME,
            embedding_function=embeddings,
            persist_directory=CHROMA_DIRECTORY
        )

    return vector_store


vector_store = initialize_vector_database()

retriever = vector_store.as_retriever(
    search_kwargs={"k": 4}
)

web_search = DuckDuckGoSearchRun()


class AgentState(TypedDict):
    question: str
    plan: str
    next_action: str
    retrieved_context: str
    tool_result: str
    draft_answer: str
    review: str
    revision_count: int
    approved: bool
    final_answer: str


def get_text(response):

    if isinstance(response.content, str):
        return response.content

    return str(response.content)


def planner_node(state: AgentState):

    print("\n[PLANNER]")

    prompt = f"""
You are a planning agent.

User question:
{state['question']}

Create a short plan for answering the question.

Possible capabilities:
1. Internal document / RAG search
2. Current web search
3. Mathematical calculation
4. Direct LLM reasoning

Return a concise step-by-step plan.
"""

    response = llm.invoke(prompt)
    plan = get_text(response)

    print("\nPlan:")
    print(plan)

    return {
        "plan": plan,
        "revision_count": state.get("revision_count", 0)
    }


def decision_node(state: AgentState):
```

```
    print("\n[DECISION]")

    prompt = f"""
You are a routing agent.

User question:
{state['question']}

Plan:
{state['plan']}

Choose the best next action.

Available actions:
rag
web
calculator
direct

Rules:
rag = internal company documents, policies, manuals or private knowledge
web = recent, latest, current or external Internet information
calculator = mathematical calculations
direct = answer without another tool

Return ONLY one word:
rag
web
calculator
direct
"""

    response = llm.invoke(prompt)
    decision = get_text(response).strip().lower()

    if "calculator" in decision:
        decision = "calculator"
    elif "rag" in decision:
        decision = "rag"
    elif "web" in decision:
        decision = "web"
    else:
        decision = "direct"

    print(f"Selected action: {decision}")

    return {"next_action": decision}


def action_router(state: AgentState):
    return state["next_action"]


def rag_node(state: AgentState):

    print("\n[RAG SEARCH]")

    documents = retriever.invoke(state["question"])

    if not documents:

        context = "No relevant information was found in the internal documents."

    else:

        parts = []

        for i, doc in enumerate(documents, start=1):

            source = doc.metadata.get("source", "Unknown")
            page = doc.metadata.get("page", "Unknown")

            parts.append(
                f"""
DOCUMENT {i}

Source:
{source}

Page:
{page}

Content:
{doc.page_content}
"""
            )

        context = "\n".join(parts)

    print("Retrieved document context.")
```

```
    return {
        "retrieved_context": context,
        "tool_result": context
    }


def web_node(state: AgentState):

    print("\n[WEB SEARCH]")

    try:
        result = web_search.invoke(state["question"])
    except Exception as e:
        result = "Web search failed: " + str(e)

    print("Web search completed.")

    return {"tool_result": result}


def calculator_node(state: AgentState):

    print("\n[CALCULATOR]")

    prompt = f"""
Extract only the mathematical expression required to answer:

{state['question']}

Example:
What is 25 multiplied by 12?

Output:
25 * 12

Return ONLY the arithmetic expression.
"""

    response = llm.invoke(prompt)
    expression = get_text(response).strip()

    print(f"Expression: {expression}")

    pattern = r"[0-9+\-*/(). %]+"

    if not re.fullmatch(pattern, expression):

        result = "The generated arithmetic expression was rejected for safety."

    else:

        try:
            result = str(
                eval(
                    expression,
                    {"__builtins__": {}},
                    {}
                )
            )
        except Exception as e:
            result = "Calculation failed: " + str(e)

    print(f"Result: {result}")

    return {"tool_result": result}


def direct_node(state: AgentState):

    print("\n[DIRECT REASONING]")

    prompt = f"""
Answer the following question clearly and accurately.

Question:
{state['question']}
"""

    response = llm.invoke(prompt)

    return {"tool_result": get_text(response)}


def agent_node(state: AgentState):

    print("\n[ANSWER GENERATOR]")

    prompt = f"""
You are the answer generation agent.
```

```
User Question:
{state['question']}

Plan:
{state['plan']}

Evidence / Tool Result:
{state['tool_result']}

Create a clear and concise answer.

Do not invent facts.
If evidence is insufficient, clearly say so.
"""

    response = llm.invoke(prompt)
    answer = get_text(response)

    print("\nDraft Answer:")
    print(answer)

    return {"draft_answer": answer}


def reviewer_node(state: AgentState):

    print("\n[REVIEWER]")

    prompt = f"""
You are an independent reviewer.

USER QUESTION:
{state['question']}

DRAFT ANSWER:
{state['draft_answer']}

AVAILABLE EVIDENCE:
{state['tool_result']}

Check:
1. Correctness
2. Relevance
3. Completeness
4. Unsupported claims
5. Whether the answer addresses the question

If acceptable, return exactly:
APPROVED

Otherwise return:
REVISE: <short explanation>
"""

    response = llm.invoke(prompt)
    review = get_text(response).strip()

    print(f"Review: {review}")

    return {"review": review}


def review_router(state: AgentState):

    revision_count = state.get("revision_count", 0)

    if state["review"].upper().startswith("APPROVED"):
        return "human_approval"

    if revision_count >= 2:
        print("\nMaximum revision limit reached.")
        return "human_approval"

    return "revise"


def revise_node(state: AgentState):

    print("\n[REVISION]")

    count = state.get("revision_count", 0) + 1

    prompt = f"""
Improve the following answer using reviewer feedback.

QUESTION:
{state['question']}

CURRENT ANSWER:
{state['draft_answer']}
```

```
REVIEWER FEEDBACK:
{state['review']}

AVAILABLE EVIDENCE:
{state['tool_result']}

Produce only the improved answer.
"""

    response = llm.invoke(prompt)
    revised_answer = get_text(response)

    print(f"Revision #{count}")
    print(revised_answer)

    return {
        "draft_answer": revised_answer,
        "revision_count": count
    }


def human_approval_node(state: AgentState):

    print("\n[HUMAN APPROVAL REQUIRED]")

    human_decision = interrupt({
        "question": state["question"],
        "draft_answer": state["draft_answer"],
        "review": state["review"],
        "instruction": "Type approve or reject."
    })

    approved = str(human_decision).strip().lower() == "approve"

    return {"approved": approved}


def approval_router(state: AgentState):

    if state["approved"]:
        return "final"

    return "revise"


def final_node(state: AgentState):

    print("\n[FINAL ANSWER]")

    return {
        "final_answer": state["draft_answer"]
    }


builder = StateGraph(AgentState)

builder.add_node("planner", planner_node)
builder.add_node("decision", decision_node)
builder.add_node("rag", rag_node)
builder.add_node("web", web_node)
builder.add_node("calculator", calculator_node)
builder.add_node("direct", direct_node)
builder.add_node("agent", agent_node)
builder.add_node("reviewer", reviewer_node)
builder.add_node("revise", revise_node)
builder.add_node("human_approval", human_approval_node)
builder.add_node("final", final_node)

builder.add_edge(START, "planner")
builder.add_edge("planner", "decision")

builder.add_conditional_edges(
    "decision",
    action_router,
    {
        "rag": "rag",
        "web": "web",
        "calculator": "calculator",
        "direct": "direct"
    }
)

builder.add_edge("rag", "agent")
builder.add_edge("web", "agent")
builder.add_edge("calculator", "agent")
builder.add_edge("direct", "agent")

builder.add_edge("agent", "reviewer")

builder.add_conditional_edges(
    "reviewer",
```

```
    review_router,
    {
        "human_approval": "human_approval",
        "revise": "revise"
    }
)

builder.add_edge("revise", "reviewer")

builder.add_conditional_edges(
    "human_approval",
    approval_router,
    {
        "final": "final",
        "revise": "revise"
    }
)

builder.add_edge("final", END)

memory = InMemorySaver()

graph = builder.compile(
    checkpointer=memory
)

config = {
    "configurable": {
        "thread_id": THREAD_ID
    }
}


def run_agent(question):

    initial_state = {
        "question": question,
        "plan": "",
        "next_action": "",
        "retrieved_context": "",
        "tool_result": "",
        "draft_answer": "",
        "review": "",
        "revision_count": 0,
        "approved": False,
        "final_answer": ""
    }

    result = graph.invoke(
        initial_state,
        config=config
    )

    while "__interrupt__" in result:

        print("\n" + "=" * 60)
        print("HUMAN REVIEW")
        print("=" * 60)

        snapshot = graph.get_state(config)
        current_state = snapshot.values

        print("\nQUESTION:\n")
        print(current_state.get("question", ""))

        print("\nPROPOSED ANSWER:\n")
        print(current_state.get("draft_answer", ""))

        print("\nREVIEWER COMMENT:\n")
        print(current_state.get("review", ""))

        decision = input(
            "\nApprove answer? (approve/reject): "
        ).strip().lower()

        if decision not in ["approve", "reject"]:
            decision = "reject"

        result = graph.invoke(
            Command(resume=decision),
            config=config
        )

    final_answer = result.get(
        "final_answer",
        ""
    )

    print("\n" + "=" * 60)
    print("FINAL ANSWER")
    print("=" * 60)
```

```
    print(final_answer)

    return final_answer


if __name__ == "__main__":

    print("\n" + "=" * 65)
    print("LANGGRAPH AGENTIC AI DEMO")
    print(
        "Cohere + RAG + Web + Calculator + "
        "Reviewer + Human Approval"
    )
    print("=" * 65)

    print("\nType 'exit' to stop.")

    while True:

        question = input("\nYou: ")

        if question.lower() in ["exit", "quit"]:

            print("\nAgent stopped.")
            break

        run_agent(question)
```

6. How the Program Works

Node / Component	Role
Planner	Creates a short plan and identifies possible capabilities.
Decision / Router	Routes to RAG, web, calculator, or direct reasoning.
RAG	Retrieves relevant private-document chunks from Chroma.
Web Search	Retrieves current or external information using DuckDuckGo.
Calculator	Handles arithmetic after extracting a restricted expression.
Direct Reasoning	Handles questions requiring no external tool.
Answer Generator	Synthesizes the plan and tool result into a draft answer.
Reviewer / Critic	Checks correctness, relevance, completeness, and unsupported claims.
Revision	Improves the draft using reviewer feedback.
Human Approval	Pauses execution for approval or rejection.
Final Answer	Returns the approved answer.
Checkpointer	Preserves graph state so interrupted execution can resume.

7. Suggested Test Cases

Scenario	Example Question	Expected Flow
Direct reasoning	Explain machine learning in simple words.	Planner → Decision → Direct → Answer Generator → Reviewer → Human Approval → Final
RAG	How many casual leave days are employees allowed?	Planner → Decision → RAG → Answer Generator → Reviewer → Human Approval → Final
Calculator	Our company has 125 employees. Each receives ₹3,500. What is the	Planner → Decision → Calculator → Answer Generator → Reviewer →

	total cost?	Human Approval → Final
Web search	What are the latest developments in quantum computing?	Planner → Decision → Web → Answer Generator → Reviewer → Human Approval → Final
Human rejection	Reject the proposed answer during approval.	Human Approval → Reject → Revision → Reviewer → Human Approval

8. Important Teaching Notes

- LangGraph provides orchestration and state management; it is not itself the intelligence.
- Cohere performs model-based planning, routing, reasoning, answer generation, and review.
- Chroma stores vectorized document chunks for RAG.
- The reviewer introduces an evaluation loop instead of accepting the first draft.
- The human approval node demonstrates Human-in-the-Loop control.
- The checkpointer lets the workflow pause and later resume from stored state.
- The example combines deterministic graph structure with dynamic model-driven decisions.

9. From Agent to Agentic AI

A basic LLM follows Prompt → LLM → Response. A tool-using agent adds external actions. This LangGraph program adds planning, dynamic routing, retrieval, actions, evaluation, revision loops, state, and human approval.

```
LLM
 ↓
Tools
 ↓
RAG
 ↓
Agent
 ↓
State
 ↓
Planning
 ↓
Dynamic Routing
 ↓
Critic / Reviewer
 ↓
Revision Loop
 ↓
Human Approval
 ↓
Agentic AI
```

10. Practical Caution

The calculator uses a restricted eval() expression for classroom simplicity. For production systems, use a dedicated arithmetic parser or calculator tool. Human approval is particularly important before sensitive or irreversible actions.

11. Recommended Next Lab

Extend this single-agent workflow into a multi-agent LangGraph system with Researcher, RAG Specialist, Analyst, Writer, Reviewer, and Supervisor agents.